

Measuring Trust Concentration as an Architectural Property

June 2026
Skip To Secure Research
Revision 1.3
Zach Miller (NoNo)

Abstract

Modern organizations rely on systems that accumulate authority by design. Identity providers, endpoint management platforms, CI/CD pipelines, DNS, package registries, and secrets managers all sit in this group. They are coordination layers. They are trusted to act across the whole estate because that is their job.

When one of these systems is compromised, the result is often rapid, organization-wide impact. This happens even when the system is operating exactly as intended. No exploit is required. The system does what it was built to do, for the wrong person.

This kind of risk is largely invisible to the frameworks organizations already use. A company can hold SOC2, ISO, CMMC, EDR, MFA, and a SIEM, and still operate a single control plane capable of destroying everything. None of those frameworks measure that. The gap is not that the danger is unknown. The gap is that trust concentration is not treated as a thing you measure, compare, or track.

This work argues that trust concentration deserves to be a first-class architectural risk. This work presents a way to measure it directly. The model was stress-tested against its own indicators and got smaller under testing, not bigger: it started at fourteen indicators and collapsed to seven once correlation analysis showed several were measuring the same thing. The model separates two questions. How much damage can a system cause because of the authority it holds by design. And how hard is malicious use of that authority for defenders to see. This work calls these Structural Blast Radius and Opacity. The contribution is not the discovery that these systems are dangerous. Experienced architects already know that. The contribution is to propose that authority concentration is an architectural property worth measuring in its own right, alongside the properties security practice already tracks, and to give organizations a shared way to reason about it. The seven indicators are a first attempt at operationalizing that idea, not the idea itself.

1. The gap

Most of security is about making trust smaller. Least privilege. Zero Trust. Attack surface reduction. All of it tries to reduce how much trust exists so there is less to lose.

That work is necessary. It is not enough.

Trust is never zero. Modern enterprises deliberately build trust concentration because it is economically efficient. Centralized identity, orchestration, endpoint management, and deployment systems reduce operational cost precisely by concentrating authority. The same architectural property that makes these

systems valuable also makes them uniquely dangerous when compromised. This framework does not argue that such systems should not exist. It argues that their concentration of authority should be treated as a measurable architectural property rather than an implicit assumption. Some systems have to be trusted to do their job. So, there is a question security practice has historically set aside. What happens when one of those trusted systems is abused? Not how to prevent it. What the damage looks like when it lands.

We have good tools for prevention. We have limited shared language for the failure itself.

One thing to be clear about up front. This framework intentionally ignores likelihood. Security people are trained to think of risk as likelihood times impact. This model measures a form of impact only: what a system can do when its trust fails, not how probable that failure is. Likelihood is a separate question, and a real one, but it is not what this aims to measure.

Here is a sharper version of the gap. A company can pass every audit it is required to pass. SOC2. ISO 27001. CMMC. It can run EDR on every endpoint, enforce MFA everywhere, and feed a SIEM. It can still have one control plane that, if compromised, has the potential to end the company.

The danger is not hidden because it is subtle. It is hidden because nothing is pointed at it. Organizations routinely inventory assets, vulnerabilities, identities, and controls. How often do they inventory concentrations of authority?

The example

A monitoring tool changes its dashboard background after a config push. The dashboard looks different on Monday. That is a change to a trusted system.

An attacker takes over your domain controller and makes unnoticed changes. That is also a change to a trusted system.

Your tooling sees both the same way: an authorized change happened. The logs look alike. The alerts, if any fire, look alike.

These are not the same event. One is a Tuesday. The other is the company going into crisis mode. Both were authorized. Both were logged. The difference is structural: what each system can do when its trust is turned against you, and how hard that use is to see.

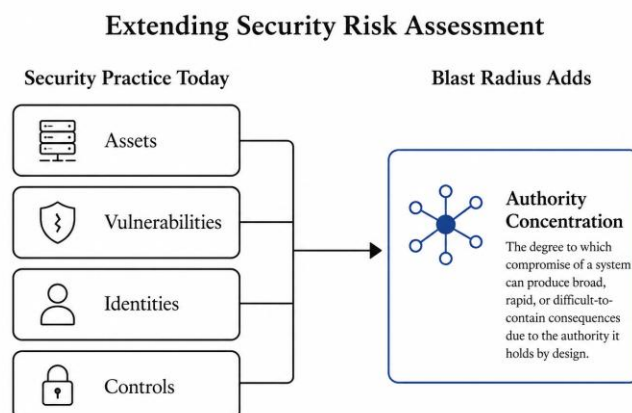


Figure 1. Authority concentration as a missing dimension in security risk assessment.

2. Two axes

The model separates two things most frameworks blend together.

Structural Blast Radius

The degree to which compromise of a system can produce broad, rapid, or difficult-to-contain consequences due to the authority the system possesses by design.

The key words are "by design." This is not about misconfiguration. A domain controller that passes every audit is still a domain controller. The danger survives a clean review. The goal here is to measure what the system can do when it is working correctly, and the wrong person is driving.

This is also what separates the model from simply ranking how much power a system has. One of the five indicators inside this axis is Irreducible Authority. It asks how much of a system's authority is unavoidable even in a perfectly clean, fully engineered deployment. Some authority is optional and can be scoped away. Some is architecturally required for the system to do its job at all. An identity provider must be trusted to assert identity. A configuration manager must be trusted to run code on its fleet. You cannot configure that away without breaking the product. That is the danger that survives a clean audit, and it is the part existing severity and tiering models do not capture. Two systems can hold the same raw power, and the one whose power is irreducible is the harder problem.

This is also why the model does not score configuration, compliance, or likelihood. Consider two deployments of the same orchestration platform. One uses MFA, separation of duties, change approvals, and full logging. The other uses none of them. The first is clearly more secure. But both deployments hold the same underlying authority to change systems at scale. Security controls reduce the likelihood that authority will be abused. They do not reduce the amount of authority present. This model measures the structural consequences of that authority, not the quality of its implementation.

Opacity

The degree to which malicious use of a system's authority is difficult for defenders to distinguish, attribute, or reconstruct.

When the dashboard changes, you notice. When an attacker uses domain admin to issue a valid-looking ticket, it looks exactly like normal work. There is no malware. There is no broken rule. The action is signed, authorized, and policy-valid. Opacity is how hard it is to tell that apart from legitimate use, and how hard it is to figure out who did it after.

Why two and not one

These answer different questions, and the difference is not academic. Blast radius is how much damage a system can do. Opacity is whether you will see it happen and how long you have to respond. The amount of authority a system holds does not change based on how well it is logged. A control plane that can reach thirty thousand endpoints has that reach whether its actions are fully audited or completely silent. Logging does not shrink the blast radius. It changes whether the blast is something you can catch in progress or something you reconstruct afterward, if at all.

That is why opacity is not a second measure of danger. It is a modifier on the danger blast radius already describes. Two systems with the same blast radius are not the same risk if one is observable and the other is not. This is also why collapsing the two into a single score would lose the information that

matters most. A single number cannot tell you whether to invest in partitioning or in telemetry. The two axes kept apart can.

3. The quadrant

Stack the two axes and you get four kinds of systems. This is the part a practitioner actually uses.

	Low Opacity (visible)	High Opacity (blind)
High Blast Radius	Scary but governable. Add approvals, partitioning, recovery paths.	Highest-priority architectural concern. Can rewrite reality before defenders observe it. Telemetry first, then partitioning.
Low Blast Radius	Boring. Baseline control.	Annoying, not existential. Do not over-invest.

The top-right cell is the one that matters most. High blast radius and high opacity. The system can do enormous damage and you will not know until it is done. Endpoint management platforms and identity providers commonly fall in this quadrant, not as a claim about any specific product, but because that class of system holds broad authority by design and its legitimate use is hard to tell from abuse.

This is the practical payoff. The purpose of the model is not to find vulnerabilities. It is to find where architectural investment produces the greatest reduction in systemic risk. A high-blast, high-opacity system is a different kind of problem than the vulnerability of the week. It needs a different kind of response, and the quadrant tells you which. Different cells get different investment. That is a decision the model makes for you that a vulnerability count cannot.

4. What this is for

A seasoned architect does not need this model to know that Okta, SCCM, Terraform org admin, and a DNS root zone are dangerous. They will look at a high score and think, yes, obviously. They are not wrong.

The model does not exist to teach experts what they already know. It exists to give organizations a shared way to reason about a risk that their existing frameworks do not capture.

The closest familiar comparison is CVSS. This work is not a replacement for vulnerability scoring systems such as CVSS, and it is not "CVSS for trust." CVSS provides a common language for discussing the technical severity of vulnerabilities. It asks what happens if this weakness is exploited. This model asks a different question. What happens if this trusted authority is compromised. A system may have no known vulnerabilities, pass every audit, and still represent a major source of systemic risk because of the authority it holds by design. That is the category this model addresses.

CVSS is useful for more than its score. Much of its value is the social function. It gave people a shared landmark so a severity conversation could happen at all. This model is trying to do the same thing for trust concentration. Consider a design review. You say "Jenkins has too much authority." The honest reply is "compared to what." Without a shared way to express it, the claim is a judgment call, and

judgment calls lose to whoever owns the system. Now you say "Jenkins scores high on structural blast radius: broad authority, low partitionability, and it acts on its own." Someone can still disagree. But now they have to say where. Is the authority not that broad. Can it be partitioned after all. The disagreement moves from whether your concern is legitimate to which specific input is wrong, and that is a question evidence can settle.

There is a reaction this work invites from experienced practitioners: "this is cute, I already know SCCM is dangerous." That reaction is correct, and it misses the point. Being able to recognize a dangerous system was never the gap. The gap is everything that comes after recognition.

The cost of that gap is not hypothetical. "I already know it's dangerous" is a comfortable position right up until the system in question is the one that fails, and then the same knowledge that felt obvious becomes a question no one can answer: if everyone knew, why are we here? Recognition that is never written down, compared, or reported is not protection. It is intuition that lives in a few people's heads and leaves when they do. The purpose of this work is to move that intuition into something an organization can hold, rank, and act on, rather than something it rediscovers after an incident.

5. The indicators

The two axes are not scored directly. They are made of smaller, concrete indicators that a person can actually evaluate. Raters score the indicators. The axis position is derived from them.

Structural Blast Radius is built from five indicators. Authority Magnitude. Irreducible Authority. Total Plane Reach. Native Partitionability. Autonomous Actuation.

Opacity is built from two. Operational Discriminability. Provenance Gap.

The five Structural Blast Radius indicators:

- **Authority Magnitude.** Native action capability, taking into account both how deep the privilege runs and how far a single action reaches.
- **Irreducible Authority.** How much of that authority is unavoidable even in an ideal, fully engineered deployment. The structural minimum, not the configured maximum.
- **Total Plane Reach.** The aggregate breadth of targets the system governs. Reach without authority is the low-danger case, so this is scored separately from authority.
- **Native Partitionability.** Whether the normal, recommended operating mode forces authority to collapse into one principal, permits compartmentalization, or enforces partition. Scored as the system is actually run, not as it could theoretically be configured.
- **Autonomous Actuation.** Whether the system acts on its own, on a schedule or in response to events, so that an attacker can ride existing automation rather than trigger anything.

The two Opacity indicators:

- **Operational Discriminability.** How indistinguishable a malicious authorized action is from legitimate operation, including whether a stable baseline of normal even exists. Higher means harder for a defender.
- **Provenance Gap.** How far telemetry fails to attribute the actor or origin behind an action. Higher means harder for a defender.

Full anchors and scoring guidance live in the instrument.

One indicator, Recovery Dependency, is carried as provisional. It asks whether the system is also the path to recover from its own compromise. It is scored but not yet assigned to an axis.

6. How the indicators were chosen

This is the part that separates a measurement from an opinion.

Every indicator has to pass two tests to stay. This is called this the constitution.

First, the independent lever test. Can the property be improved without materially changing something the instrument already scores. If the only way to fix it is to change another indicator, it is not its own thing. It is a symptom.

Second, the architectural property test. Is it a structural fact about the system, not an organizational or maturity fact. This blocks things like training and runbooks. Those matter, but they are not properties of the system.

An indicator has to pass both. Passing one is not enough.

The graveyard

This started with fourteen indicators. I did not keep them because I liked them. They were tested, and I cut the ones that failed.

Indicator	Disposition	Reason
Privilege Depth	Folded into Authority Magnitude	Correlated near 0.98 with Fan-out. The signal is preserved as an input to Authority Magnitude rather than a separate column.
Single-action Fan-out	Folded into Authority Magnitude	Correlated near 0.98 with Privilege Depth. Preserved as the reach dimension of the Authority Magnitude anchor.
Propagation Latency	Folded into the derived output	Correlated near 0.90 with Fan-out. Feeds the derived Defender Gap, not a standalone column.
In-flight Reversal Window	Folded into the derived output	Correlated near 0.90 with Defender Gap. An input to the derived output, not its own dimension.
Defender Gap	Removed as a scored column, kept as a derived output	Did not satisfy the constitution under current evidence. Every way to improve it routed back to authority or telemetry.

		Correlated 0.91 with Authority Magnitude. Reopenable if an independent lever is ever found.
--	--	---

This started with fourteen indicators and kept seven, plus one derived output and one provisional column.

I decided to show the dead indicators on purpose. It answers a fair criticism before it is made. Did you remove things when the data disagreed with your intuition. Yes. Here is the list.

7. Testing our own work

A model that only makes sense to the people who built it is not validated. It is just coherent.

I ran a correlation analysis on the early scores with the help of some peers. The result was uncomfortable and useful. One factor explained most of the variance. Fourteen indicators were measuring about three or four things. That is false precision. There were columns that looked independent and were not.

So, they were collapsed. Two indicators that correlated at 0.98 became one. A modifier that correlated with authority at 0.91 was removed as a scored column and turned into a derived output. The model went from fourteen columns to seven plus one derived output. It got smaller under testing, not bigger.

To be specific about the numbers. In the first analysis, one factor explained about 73 percent of the variance across the indicators, and roughly four components covered 90 percent. Fourteen columns were carrying three to four real dimensions. Several pairs were nearly collinear: Privilege Depth and Fan-out at 0.98, and Defender Gap with Authority Magnitude at 0.91. I then scored a wider, deliberately varied set of systems to spread the range, and the strongest collinearity relaxed, which is what separated the genuinely independent indicators from the redundant ones. The independent survivors, including Provenance Gap, Autonomous Actuation, Total Plane Reach, and Partitionability, held their own signal across both rounds.

There has been a deliberate effort to avoid claiming a precise danger formula. The interaction between the two axes is real, but whether it is additive, multiplicative, or something else is an open question. I leave the exact equation to a larger, subsequent study rather than invent one I cannot defend this early on.

8. Validation

At the time of this paper's publishing, further validation is taking place. An independent scoring trial is underway in which raters who did not build the model score a set of systems from reputation-stripped descriptions. Results will be reported separately.

9. The pattern in the wild

Look at the last few years.

SolarWinds. Kaseya. Okta. CircleCI. Stryker.

This model was partly motivated by the observation that incidents such as these appear to share a structural pattern. I have tended to file them under supply chain. That label may be hiding what they have in common. In each one, a trusted coordination layer was the weapon. The thing trusted to run everything got turned, and it ran everything for the attacker instead. Getting to that layer sometimes took an exploit. Using it did not. Legitimate functionality. Trusted vendor pipeline. Broad reach.

If that hypothesis holds, these are not a string of unrelated incidents. They are the same shape repeating. A trusted plane, high blast radius, high opacity, used exactly as designed against the people who deployed it. Testing whether that shape is real and measurable is part of what this model is for. I state it as a hypothesis on purpose. The model is the thing meant to test it, not the proof that it is true.

The SCCM attack chain studied in the management-plane work referenced below is a concrete case in the high blast radius, high opacity quadrant. The system holds broad authority by design. A single network access account credential extracted from it exists on every managed endpoint, and a relay path leads to full administrative control of the site hierarchy. That is the blast radius. The opacity was measured directly. Logs from the full attack chain were evaluated against the complete community Sigma ruleset, 2,886 rules in total. Of eleven distinct attack steps, only four produced any detection, and all four were generic patterns unrelated to SCCM. The seven steps specific to the management plane were invisible. After writing six custom rules, coverage of the steps that produce evidence in default logging rose from forty-four percent to seventy-eight percent. The remaining gap was a log source problem, not a rule problem: the relevant events are not recorded in default Windows logging or required Sysmon. This is great example of what high opacity looks like in practice. The authority operates as designed, the abuse is indistinguishable from legitimate use, and the telemetry needed to tell them apart is off by default.

10. Threats to validity

This is early work and it has real limits. Naming them is part of the method.

Single-rater testing influenced early development. Much of the indicator refinement happened against a few rater's scores before independent scoring began. That is a source of bias and it shaped the model.

System selection was intentional, not random. The systems scored so far were chosen to stress the model, including deliberate edge cases. That is useful for development and it is not a representative sample.

Validation has not been completed. As of this draft, the instrument is frozen for a first validation round that has not yet run. Claims of external validity are not yet earned.

The model may not generalize past enterprise control planes. It was built from management-plane and identity examples. Whether the indicators behave sensibly on very different kinds of systems is an open question.

There is also an honest tension worth stating. A seasoned architect may not need this model personally. They can already see the danger. The argument is that organizations need it collectively, the way risk disciplines like aviation and reliability engineering use shared language so that concerns become discussable and trackable. That is a claim about organizational value, not personal insight, and it should be read that way.

11. Future work: composed trust

This work intentionally evaluates authority-bearing systems in isolation. Modern enterprise environments increasingly compose trusted systems into automated chains. A source code commit may trigger a CI/CD pipeline, which invokes an orchestration platform, which modifies cloud infrastructure. In these environments, authority can propagate across multiple trusted systems through legitimate delegation rather than through direct compromise of the final target.

The property that matters here is not position in the chain. It is unbroken delegation. If a manual approval sits between two systems, the upstream system has not inherited the downstream system's authority. If the path is fully automatic, it has. What matters is whether one trusted system can legitimately cause another to exercise its authority. Delegation of that kind can run upstream, downstream, or laterally, which is why it is a different construct from the node-level blast radius measured here.

The opacity problem is sharper in composed paths than in any single system. Each system on the path sees a legitimate request. The source control system sees a valid commit. The pipeline sees a valid trigger. The orchestration platform sees a valid playbook. No system is lied to. The malicious intent exists only in the composition, which is exactly why instrumenting each system in isolation does not surface it.

Measuring delegated authority across connected trust relationships is outside the scope of the current instrument. It is a known limitation and a direction for future work. Incidents such as SolarWinds and CircleCI illustrate that authority frequently propagates through chains of trusted systems rather than terminating at a single platform, which suggests these composed trust paths are an important extension of the structural risk model introduced here.

12. What this is and is not

This is a way to measure a kind of risk that compliance, control, and vulnerability frameworks do not capture. It tells you which of your trusted systems would end you if they broke, and whether you would see it happening.

It is not a vulnerability scanner. It does not find bugs. It measures structure.

It is not a replacement for least privilege or Zero Trust. Those reduce trust. This measures what is left after you have reduced all you can.

This work is not finished. The core claim is simple. Some trust has to exist. So the real question is not only how do we trust less. It is which of our trusted systems would level us if they broke, and would we even see it. The larger proposal underneath that question is that authority concentration deserves to be measured as an architectural property in its own right. This work is a first attempt at that measurement, not the last word on it.

Every organization has systems it cannot live without and cannot fully see. Most can name them. Few can measure them, rank them against each other, or show whether the risk is getting better or worse. That gap between knowing and measuring is where this work lives. The danger was never that the risk is unknown. It is that it is untracked.

Citations

1. Common Vulnerability Scoring System (CVSS), version 4.0 Specification. Forum of Incident Response and Security Teams (FIRST). <https://www.first.org/cvss/>
2. Kindervag, J. No More Chewy Centers: Introducing the Zero Trust Model of Information Security. Forrester Research, 2010.
3. Rose, S., Borchert, O., Mitchell, S., and Connelly, S. NIST Special Publication 800-207: Zero Trust Architecture. National Institute of Standards and Technology, August 2020. <https://csrc.nist.gov/pubs/sp/800/207/final>
4. Miller, Z. LOLMgmt: Living Off the Land in the Management Plane. NolaCon 2026. https://github.com/ridgestoneco/LOLMgmt_NolaCon2026
5. Cybersecurity and Infrastructure Security Agency. Kaseya VSA Supply-Chain Ransomware Attack. CISA Current Activity, July 2, 2021. <https://www.cisa.gov/news-events/alerts/2021/07/02/kaseya-vsa-supply-chain-ransomware-attack>
6. CircleCI. Incident Report for January 4, 2023 Security Incident. January 13, 2023. <https://circleci.com/blog/jan-4-2023-incident-report/>
7. Okta. Unauthorized Access to Okta's Support Case Management System. Okta Security, October 2023. <https://sec.okta.com/articles/2023/10/unauthorized-access-oktas-support-case-management-system/>
8. Microsoft Security Response Center. Microsoft Internal Solorigate Investigation Final Update. February 18, 2021. <https://msrc.microsoft.com/blog/2021/02/microsoft-internal-solorigate-investigation-final-update/>
9. Microsoft. Securing Privileged Access: Enterprise Access Model. Microsoft Learn. <https://learn.microsoft.com/en-us/security/privileged-access-workstations/privileged-access-access-model>
10. SpecterOps. Misconfiguration Manager: Overlooked and Overprivileged. June 2025. <https://specterops.io/blog/2025/06/26/misconfiguration-manager-still-overlooked-still-overprivileged/>
11. International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC). *ISO/IEC 27001:2022 – Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. Geneva, Switzerland: ISO, 2022